

17 · Quick Revision Cheat Sheet

The whole set on a few pages. Read it the evening before — every line points back to a chapter (§ = chapter number) if something is hazy.

1 · Command families (§01)

	Commands	Notes
DDL	CREATE ALTER DROP TRUNCATE RENAME	structure · auto-commits (Oracle, MySQL)
DML	INSERT UPDATE DELETE (+ SELECT in some books)	data · needs COMMIT , can ROLLBACK
DQL	SELECT	read-only
DCL	GRANT REVOKE	access
TCL	COMMIT ROLLBACK SAVEPOINT SET TRANSACTION	transactions

2 · Oracle data types (§01)

Need	Type	Remember
text	VARCHAR2(n) ≤ 4000 · CHAR(n) ≤ 2000 (padded) · CLOB	never VARCHAR
number	NUMBER(p,s) — p-s digits before the point; negative s rounds left	INTEGER = NUMBER(38)
date	DATE has time · TIMESTAMP (fractions) · ... WITH [LOCAL] TIME ZONE	SYSDATE
binary	BLOB · BFILE (outside DB, ≤ 4 GB)	
boolean	PL/SQL only (NUMBER(1) in SQL before 23ai)	

NULL ≠ 0 ≠ '' — except **Oracle: '' is NULL**. Three-valued logic: a WHERE keeps only TRUE. COUNT(*) counts rows; COUNT(col) skips NULLs; SUM/AVG/MIN/MAX ignore NULLs.

3 · DDL (§02)

```
CREATE TABLE t (id NUMBER PRIMARY KEY, name VARCHAR2(50) NOT NULL, born DATE DEFAULT SYSDATE);
CREATE TABLE copy AS SELECT * FROM t;                                -- structure + data    (WHERE 1=0 → structure
only)
ALTER TABLE t ADD (email VARCHAR2(100));                            -- add
ALTER TABLE t MODIFY (name VARCHAR2(80));                          -- change type/size   (SQL Server: ALTER
COLUMN)
ALTER TABLE t RENAME COLUMN name TO full_name;                    -- rename column
ALTER TABLE t DROP COLUMN email;                                    -- drop column
RENAME t TO t2;                                                      -- rename table
```

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	some/all rows	all rows	the table
WHERE	✓	✗	✗
Rollback	✓	✗	✗ (recycle bin → FLASHBACK)
Triggers	fires	no	no

Auto ids: MySQL `AUTO_INCREMENT` · SQL Server `IDENTITY` · PostgreSQL `SERIAL` · **Oracle** `GENERATED ALWAYS | BY DEFAULT [ON NULL] AS IDENTITY` (12c+) or `SEQUENCE.NEXTVAL` . Deleting rows never resets a counter; gaps are normal.

4 · Constraints (\$03)

	Rule
NOT NULL	value required
UNIQUE	no duplicates; many per table; NULLs allowed
PRIMARY KEY	NOT NULL + UNIQUE ; one per table; may be composite
FOREIGN KEY	must match a parent PK/UNIQUE; may be NULL; ON DELETE CASCADE / SET NULL (Oracle has no ON UPDATE)
CHECK	condition on the row — NULL passes ; no SYSDATE /subqueries
DEFAULT	used only when the column is omitted; not a constraint (can't be named)

Errors: ORA-00001 duplicate · 01400 NULL · 02290 check · **02291 no parent** · **02292 has children**. Name them: `CONSTRAINT pk_emp PRIMARY KEY` . Oracle doesn't auto-index FKs.

5 · SELECT skeleton and order of execution (\$04)

```

SELECT [DISTINCT] cols/exprs           -- 5
FROM   t1 [JOIN t2 ON ...]             -- 1
WHERE  row-filter                      -- 2
GROUP  BY cols                         -- 3
HAVING group-filter                    -- 4
ORDER  BY cols [ASC|DESC] [NULLS FIRST|LAST]  -- 6
FETCH  FIRST n ROWS ONLY;              -- 7

```

Topic	Rule
Precedence	NOT > AND > OR — parenthesise
IN	shorthand for many OR s · BETWEEN inclusive · LIKE : % any, _ one
[] in LIKE	not Oracle → REGEXP_LIKE(col, '^[AR]')
NULL test	IS [NOT] NULL , never = NULL

Topic	Rule
Aliases	column: <code>AS</code> optional; table: no AS in Oracle ; not usable in <code>WHERE</code>
Top-N	<code>FETCH FIRST n ROWS ONLY</code> · old: sort in subquery, then <code>ROWNUM <= n</code>
CASE	<code>CASE WHEN c THEN v ... ELSE d END</code> — no <code>ELSE</code> $\Rightarrow$ NULL

6 · DML & transactions (§05)

```

INSERT INTO t (a,b) VALUES (1,'x');
INSERT ALL INTO t (a,b) VALUES (1,'x') INTO t (a,b) VALUES (2,'y') SELECT * FROM dual;  -- Oracle
multi-row / multi-table
INSERT INTO t2 SELECT * FROM t1 WHERE ...;
UPDATE t SET a = a * 1.1 WHERE ...;      DELETE FROM t WHERE ...;      -- no WHERE  $\Rightarrow$  ALL rows
COMMIT;  ROLLBACK;  SAVEPOINT s1;  ROLLBACK TO s1;

```

DDL **implicitly commits**. Oracle before 23ai has no multi-row `VALUES (...), (...)`.

7 · Aggregation (§06)

Functions	<code>COUNT SUM AVG MIN MAX (MEDIAN)</code>
<code>GROUP BY</code> rule	every non-aggregated <code>SELECT</code> column must be grouped (ORA-00979)
WHERE vs HAVING	rows before grouping (no aggregates) vs groups after aggregation
Duplicates	<code>GROUP BY c HAVING COUNT(*) > 1</code>
Empty groups	<code>LEFT JOIN + COUNT(child.col)</code>
Pivot	<code>SUM(CASE WHEN ... THEN 1 ELSE 0 END)</code>
Subtotals	<code>ROLLUP(a,b) = (a,b),(a,())</code> · <code>CUBE(a,b)</code> = all combinations · <code>GROUPING()</code> marks them

8 · Joins & set operators (§07)

Join	Rows returned
<code>INNER</code>	match in both
<code>LEFT / RIGHT</code>	all of one side + matches (NULL where none)
<code>FULL</code>	all of both
<code>CROSS</code>	every combination ($m \times n$)
<code>SELF</code>	table joined to itself (two aliases)

`JOIN` = `INNER JOIN` · **ON** = **how to join**; **WHERE** = **filter after** — for outer joins a `WHERE` on the optional side turns it inner. Anti-join: `LEFT JOIN ... WHERE right.pk IS NULL` · Legacy Oracle: `WHERE a.x = b.x (+)` · `UNION` (distinct) · `UNION ALL` (keeps duplicates, faster) · `INTERSECT` · `MINUS` · Same column count/types; `ORDER BY` last.

9 • Subqueries (§08)

Shape	Use	Operator
Scalar	one value	<code>= > < (ORA-01427 if > 1 row)</code>
Column	a list	<code>IN, ANY, ALL</code>
Row	a tuple	<code>(a,b) IN (SELECT ...)</code>
Table	derived table	<code>in FROM</code>

Correlated	references the outer query; per outer row
<code>EXISTS</code>	any row? — short-circuits, NULL-safe
<code>NOT IN + NULL in list</code>	returns nothing → use <code>NOT EXISTS</code>
<code>> ANY = > MIN · > ALL = > MAX · = ANY = IN · != ALL = NOT IN · != ANY ≠ NOT IN</code>	empty subquery: <code>ALL TRUE, ANY FALSE</code>
2nd highest	<code>SELECT MAX(x) FROM t WHERE x < (SELECT MAX(x) FROM t)</code>

10 • Functions (§09)

Family	Functions
Case	<code>UPPER LOWER INITCAP</code>
Strings	<code>SUBSTR(s, start, len)</code> (negative start ok) · <code>INSTR(s, sub[, start, occ])</code> (0 = not found) · <code>LENGTH REPLACE TRANSLATE LPAD RPAD LTRIM RTRIM TRIM CONCAT(2 args) </code>
Numbers	<code>ROUND(n, d)</code> (negative d rounds left) · <code>TRUNC CEIL FLOOR MOD ABS POWER SQRT SIGN</code>
Dates	<code>SYSDATE ± n days · ADD_MONTHS · MONTHS_BETWEEN · NEXT_DAY · LAST_DAY · ROUND/TRUNC(d, 'MONTH' 'YEAR')</code> · <code>EXTRACT</code>
Convert	<code>TO_CHAR(d, fmt)</code> (FM removes padding) · <code>TO_DATE(s, fmt)</code> · <code>TO_NUMBER</code> · <code>TO_TIMESTAMP</code>
NULLs	<code>NVL(a, b) · NVL2(x, notnull, null) · NULLIF(a, b) · COALESCE(a, b, ...)</code>
Regex	<code>REGEXP_LIKE / INSTR / SUBSTR / REPLACE / COUNT</code>

Age: `TRUNC(MONTHS_BETWEEN(SYSDATE, dob)/12)` · ' in a string: `' · 'abc' || NULL = 'abc'`.

11 • Views & indexes (§10)

- **View** = stored `SELECT`, no data. `CREATE [OR REPLACE] VIEW v AS ... · DROP VIEW v`. `WITH CHECK OPTION` rejects rows leaving the view (ORA-01402); `WITH READ ONLY`. Simple views are updatable; joins/ `DISTINCT` / `GROUP BY` need `INSTEAD OF` triggers.
- **Index** = sorted B-tree → no full scan. Auto on `PRIMARY KEY / UNIQUE`. `CREATE [UNIQUE] INDEX i ON t (c1[, c2])` · `DROP INDEX i`.
- Composite: **leftmost-prefix**. Not used for: function on column, leading %, implicit conversion, small/low-selectivity tables. Too many indexes slow writes. Function-based: `ON t (LOWER(c))`.

12 · Window functions (§11)

```
fn() OVER (PARTITION BY g ORDER BY x [ROWS|RANGE ...])
```

	100, 90, 90, 80
ROW_NUMBER	1, 2, 3, 4
RANK	1, 2, 2, 4
DENSE_RANK	1, 2, 2, 3

Cannot appear in `WHERE` → compute in a subquery, filter outside. Running total: `SUM(x) OVER (ORDER BY d)` (ties share a value; use `ROWS` for row-by-row). Top-N per group: `DENSE_RANK() ... ≤ N`.

13 · PL/SQL (§12–15)

```
DECLARE    v NUMBER := 0;    c CONSTANT NUMBER := 3.14;    n emp.emp_name%TYPE;    r emp%ROWTYPE;
BEGIN      ...                                -- mandatory
EXCEPTION WHEN NO_DATA_FOUND THEN ...        -- optional
END;
/
```

Output	<code>SET SERVEROUTPUT ON;</code> · <code>DBMS_OUTPUT.PUT_LINE(x)</code>
Assign / compare	<code>:=</code> / <code>=</code>
Control	<code>IF ... ELIF ... ELSE ... END IF</code> · <code>CASE ... END CASE</code> (needs <code>ELSE</code>) · <code>LOOP ... EXIT WHEN</code> · <code>WHILE ... LOOP</code> · <code>FOR i IN [REVERSE] a..b LOOP</code>
<code>SELECT INTO</code>	exactly 1 row: <code>NO_DATA_FOUND</code> / <code>TOO_MANY_ROWS</code>
NULL in <code>IF</code>	UNKNOWN → <code>ELSE</code> branch
<code>VARRAY</code>	1-based, bounded, initialise with constructor

Subprograms

	Procedure	Function
Returns	via <code>OUT</code>	<code>RETURN</code> (exactly one)
Call	<code>EXEC p(x)</code> / <code>BEGIN p(x); END;</code>	expression / <code>SELECT f(x) FROM dual</code>

Parameter modes: `IN` (default, read-only) · `OUT` (starts NULL) · `IN OUT`. Notation: positional · named `=>` · mixed (positional first). Header: `CREATE OR REPLACE PROCEDURE p (a IN NUMBER) IS ... BEGIN ... END;` — **no `DECLARE`**, no length on parameter types. `OUT` from SQL*Plus: `VARIABLE x NUMBER; EXEC p(:x); PRINT x`. **Package** = spec (public) + body (implementation, private items); call `pkg.proc`.

Cursors

	Implicit (SQL)	Explicit
Steps	automatic	CURSOR c IS ... → OPEN → FETCH ... INTO → CLOSE
Attributes	SQL%FOUND %NOTFOUND %ROWCOUNT , %ISOPEN (always FALSE)	c%FOUND %NOTFOUND %ROWCOUNT %ISOPEN

FOR rec IN (SELECT ...) LOOP does it all. Records: t%ROWTYPE , c%ROWTYPE , TYPE ... IS RECORD (type before variable).

Triggers

```
CREATE OR REPLACE TRIGGER t {BEFORE|AFTER|INSTEAD OF} {INSERT|UPDATE [OF c]|DELETE} ON tbl
[FOR EACH ROW] [WHEN (NEW.x > 0)] BEGIN ... END;
```

:OLD / :NEW need FOR EACH ROW (ORA-04082) · :OLD NULL on insert, :NEW NULL on delete · order: before stmt → (before row → change → after row)_n → after stmt · INSERTING/UPDATING/DELETING · reject with RAISE_APPLICATION_ERROR(-20xxx, msg) · **mutating table ORA-04091** → compound trigger · no COMMIT (ORA-04092).

14 · The errors worth knowing

Code	Meaning
ORA-00904	invalid identifier (e.g. alias used in WHERE)
ORA-00933	SQL command not properly ended (e.g. AS before a table alias)
ORA-00934	group function not allowed here (aggregate in WHERE)
ORA-00979	not a GROUP BY expression
ORA-01427	single-row subquery returns more than one row
ORA-01403 / ORA-01422	NO_DATA_FOUND / TOO_MANY_ROWS
ORA-01438	value larger than precision allowed
ORA-12505	listener doesn't know the SID (use Service name)
ORA-04091 / 04082 / 04092	mutating table / :NEW - :OLD at statement level / commit in trigger

15 · Ten answers to have ready

1. **DELETE vs TRUNCATE vs DROP** — rows (DML, rollback) / all rows (DDL, fast) / whole table.
2. **WHERE vs HAVING** — rows before grouping / groups after aggregation.
3. **INNER vs LEFT JOIN**, and *customers with no orders* → LEFT JOIN ... IS NULL or NOT EXISTS .
4. **Why NOT IN returns nothing** — a NULL in the list makes every comparison UNKNOWN.
5. **RANK vs DENSE_RANK vs ROW_NUMBER** — gap, no gap, unique.
6. **Second-highest salary** — DENSE_RANK() = 2 , or MAX(x) WHERE x < (SELECT MAX(x)) .
7. **Primary vs unique key** — one, no NULL / many, NULL allowed.
8. **Procedure vs function; IN / OUT / IN OUT**.
9. **Implicit vs explicit cursor; row- vs statement-level trigger**.

10. **The order of execution** — FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → FETCH.